

AUTONOMOUS VEHICLE ROUTE SIMULATOR

A DAA-Based Route Planning and Visualization Project

Problem No.: 64

Core Algorithms: BFS, Dijkstra's Algorithm, A* Search

Suggested Technology: Python, Pygame

Project Type: Algorithm-Centric Prototype

1. Abstract

The Autonomous Vehicle Route Simulator is an algorithm-centric prototype designed to demonstrate how route-planning algorithms can guide a vehicle through a grid containing obstacles and roads with different traversal costs. The system provides an interactive visualization of Breadth-First Search (BFS), Dijkstra's Algorithm, and A* Search. Users can define a starting point and destination, place obstacles, assign changing road costs, select an algorithm, and observe the search process and final vehicle movement. The project emphasizes not only the final route but also the decisions made by each algorithm, including explored nodes, path cost, path length, and search behavior.

2. Introduction

Autonomous vehicles need reliable route-planning methods to move from one location to another while avoiding obstacles and responding to changing road conditions. A route that is shortest in physical distance may not always be the least costly route when traffic, road conditions, or other weights are considered. This project models that problem as a graph-search problem and provides a visual environment for understanding how different algorithms solve it.

The simulator represents a road network as a grid. Each cell represents a location or road segment, and neighboring cells form graph connections. Obstacles remove available connections, while weighted cells represent different traversal costs. The system then uses BFS, Dijkstra, or A* to find an appropriate route.

3. Problem Statement

Simulate vehicle movement through a grid or road graph containing obstacles and changing edge or cell costs. The system must calculate a route from a selected source to a destination and make the algorithm's important decisions visible to the user.

4. Objectives

- Implement BFS, Dijkstra's Algorithm, and A* Search for route planning.
- Represent the environment using a graph/grid data structure.
- Allow users to create obstacles and assign different traversal costs.
- Visualize the nodes explored by the selected algorithm.
- Reconstruct and display the final route.
- Animate vehicle movement along the calculated route.
- Compare algorithm behavior using path length, total cost, explored nodes, and execution time.
- Demonstrate how changing road costs can cause a route to be recalculated.

5. Scope of the Project

- Grid-based simulation of an autonomous vehicle.
- Static and dynamically changing obstacles/costs.
- Three classical graph-search algorithms.
- Interactive visualization of search and movement.
- Algorithm comparison for educational and demonstration purposes.
- The prototype does not attempt to control a real vehicle or replace a production navigation system.

6. Proposed System

The proposed system consists of a grid editor, algorithm engine, route reconstruction module, visualization layer, vehicle simulator, and performance comparison panel.

6.1 Main Components

- Grid Manager – creates and maintains the road grid.
- Obstacle Manager – marks cells as blocked or traversable.
- Cost Manager – assigns traversal costs such as normal, traffic, and heavy-traffic costs.
- Algorithm Engine – executes BFS, Dijkstra, or A*.
- Path Reconstruction – uses parent/predecessor information to recover the final route.
- Visualization Engine – displays explored states and the final path.
- Vehicle Simulator – animates the vehicle from source to destination.
- Performance Analyzer – records path length, total cost, explored nodes, and execution time.

7. Data Structures

- 2D list/grid for representing the environment.
- Queue for BFS.
- Priority queue/min-heap for Dijkstra and A*.
- Set for visited/explored nodes.
- Dictionary or array for tentative distances/costs.
- Parent/predecessor mapping for path reconstruction.

8. Algorithms Used

8.1 Breadth-First Search (BFS)

BFS explores nodes level by level using a queue. In an unweighted grid where every movement has the same cost, BFS can find a path with the minimum number of moves. It is useful for demonstrating reachability and minimum-hop routing.

Time Complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges.

Space Complexity: $O(V)$.

8.2 Dijkstra's Algorithm

Dijkstra's Algorithm calculates the least-cost path when all edge weights are non-negative. It maintains a tentative cost for each node, repeatedly selects the unexplored node with the smallest tentative cost, and relaxes its neighboring edges. A priority queue is used to efficiently select the next node.

Time Complexity with a binary heap: $O((V + E) \log V)$.

Space Complexity: $O(V)$.

8.3 A* Search

A* combines the cost already travelled with a heuristic estimate of the remaining distance to the destination. For a four-directional grid, Manhattan distance is a suitable heuristic when movement costs are compatible with the model. A* generally focuses exploration toward the selected destination rather than expanding uniformly across the grid.

Typical time complexity depends on the heuristic and graph structure; in practice, a good admissible heuristic can substantially reduce explored states.

Space Complexity: $O(V)$.

9. Algorithm Comparison

Algorithm	Weighted Costs	Goal Direction	Main Data Structure	Typical Use
BFS	No	No	Queue	Minimum-hop path
Dijkstra	Yes	No	Priority Queue	Least-cost path
A*	Yes	Yes	Priority Queue	Goal-directed least-cost path

10. System Workflow

- Create the grid environment.
- Select the source and destination cells.
- Add obstacles and assign road/traversal costs.
- Select BFS, Dijkstra, or A*.
- Start the algorithm visualization.
- Display explored nodes and intermediate search states.
- Reconstruct the route when the destination is reached.
- Display route length, total cost, explored nodes, and execution time.
- Animate the autonomous vehicle along the selected route.
- Optionally change a road cost or obstacle and recalculate the route.

11. User Interface Design

- Grid Area – primary area for editing and visualizing the road network.
- Control Panel – algorithm selection, reset, start, and simulation controls.
- Legend – explains start, destination, obstacles, explored nodes, and final path.
- Cost Controls – allow users to assign different traversal costs.
- Statistics Panel – displays algorithm metrics.
- Vehicle Animation – visually demonstrates movement along the computed path.

12. Suggested Color/State Representation

- Green: Starting position.
- Red: Destination.
- Black/Dark: Obstacle or blocked road.
- Yellow: Node currently being explored.
- Light Blue: Previously explored node.
- Blue: Final calculated route.
- Vehicle icon: Current autonomous vehicle position.

13. Dynamic Route Recalculation

A key feature of the simulator is changing road conditions. For example, a road initially having cost 1 may be changed to cost 7 to represent heavy traffic. Dijkstra or A* can then be executed again to determine whether another route has a lower total cost. This demonstrates why route planning must consider more than geometric distance in a weighted environment.

14. Performance Metrics

- Execution time – time required to calculate the route.
- Nodes explored – number of grid cells examined by the algorithm.
- Path length – number of movements/cells in the final route.
- Total path cost – sum of traversal costs along the selected route.
- Search efficiency – comparison of explored states between algorithms.

15. Expected Results

The simulator is expected to produce a valid route whenever the destination is reachable. BFS should demonstrate minimum-hop behavior in an unweighted grid. Dijkstra should account for varying traversal costs and identify a least-cost route. A* should use its heuristic to focus exploration toward the destination while maintaining optimality when the heuristic is appropriate for the problem. When road costs change, the displayed route may also change.

16. Advantages

- Highly visual and easy to demonstrate.
- Directly applies important DAA concepts.
- Supports both weighted and unweighted route planning.
- Makes algorithm decisions visible instead of showing only the final answer.
- Allows meaningful comparison of classical search algorithms.
- Can be extended with additional algorithms and simulation features.

17. Limitations

- The grid is a simplified representation of a real road network.
- The prototype does not model real-world traffic prediction.
- Vehicle physics and road rules are simplified.
- A changing environment may require repeated route calculations.
- Execution-time comparisons can vary with hardware and grid size.

18. Future Enhancements

- Real road-map graph support.
- Live traffic or simulated traffic updates.
- Multiple autonomous vehicles.
- Dynamic obstacles such as moving vehicles.
- Additional algorithms such as Greedy Best-First Search.
- Different heuristic functions for A*.

- Heatmaps for traffic and search density.
- Web-based React interface.
- Exportable algorithm performance reports.
- Multi-destination route planning.

19. Technologies Required

- Python – core implementation language.
- Pygame – interactive grid and vehicle visualization.
- Python heapq – priority queue implementation for Dijkstra and A*.
- Optional React – web-based front end if a browser version is developed.

20. Conclusion

The Autonomous Vehicle Route Simulator provides a practical and visual demonstration of fundamental graph-search algorithms used in route planning. By combining BFS, Dijkstra's Algorithm, and A* with obstacles and changing road costs, the project shows how different algorithmic strategies behave under different conditions. The visualization of explored nodes, selected routes, costs, and vehicle movement makes the underlying DAA concepts easier to understand and provides a strong foundation for further autonomous-navigation simulations.

21. References / Learning Resources

- Standard algorithms and data structures coursework covering graph traversal and shortest paths.
- Python documentation for lists, sets, dictionaries, and heap-based priority queues.
- Pygame documentation for 2D graphics, event handling, and animation.
- Classical literature and educational material on BFS, Dijkstra's Algorithm, and A* Search.